ADR-S-025: CONSENSUS — Multi-Stakeholder Evaluator Functor

Series: S (specification-level — applies to all implementations) **Status**: Accepted **Date**: 2026-03-08 **Scope**: Evaluator model extension — multi-party F_H with quorum semantics

Review record: - Proposed: Claude Code `20260308T090000` - Gemini review: `20260308T061000` — approved with `participation_threshold` addition - Codex review: `20260308T173425` — approved with gating snapshot, participation floor, asset versioning, config invariant, veto deferral - Ratified: 2026-03-08 (majority, all material findings incorporated)

Context

The current evaluator model defines three types: F_D (deterministic), F_P (agent), F_H (human). F_H is singular — one human approves or rejects. The convergence rule for an F_H check is binary: the human passed it or did not.

This is insufficient for scenarios where multiple accountable participants must evaluate an asset before it can converge:

- Spec-level proposals requiring working group agreement
- ADR acceptance by architecture committee
- Release gates requiring sign-off from multiple stakeholders
- Public review of methodology changes

These are not edge cases. They appear at the highest-stakes edges in any governance-aware methodology. Without a formal model, implementations handle multi-party evaluation ad hoc, producing inconsistent semantics across tenants.

ADR-S-024 addresses when marketplace signals become development gates. This ADR addresses a different question: how does the formal evaluator model represent N participants voting on a single asset within a single iterate() call.

Decision

CONSENSUS as a Parameterisation of F_H

CONSENSUS is a **higher-order functor** over F_H — not a new evaluator type. The formal system remains {F_D, F_P, F_H}. CONSENSUS is:

```
CONSENSUS(edge, roster, quorum, participation) = iterate(asset,
context, MultiF_H(roster, quorum, participation))
  where converged ⟺ min_duration_elapsed = true
                AND review_window_closed = true
                AND participation_threshold_met = true
                AND quorum(gating_votes) = true
                AND gating_comments_dispositioned = true
```

This preserves the four-primitive model. CONSENSUS adds no new primitive — it parameterises F_H with a roster, quorum rule, and participation floor.

Type signature (Fong & Spivak prop notation):

```
CONSENSUS : (1 + N) → Decision<ApprovedAsset, ConsensusFailure>
```

One proposal wire in, N participant wires in, one decision wire out. The codomain is a sum type — either an approved asset (carrying `stability: ratified` if paired with RATIFY) or a typed `ConsensusFailure` result.

§5.3 relationship: CONSENSUS introduces the first lower-bound constraint type in the formal system. See AI_SDLC_ASSET_GRAPH_MODEL.md §5.3 for the distinction between upper-bound (`time_box.max_duration` — forces fold-back when exceeded) and lower-bound (`min_duration` — gates convergence until elapsed). These are sensors in opposite directions. A lower-bound breach signal is `evaluation_deferred`, not `evaluation_failed` — the candidate is still valid; the window is not yet open.

Operational Semantics

Phase 1: Publication

The proposer publishes the asset to the participant roster and opens the review window:

```yaml
publication:
  asset_id: {PROP-SEQ | REQ-key | ADR-id}
  asset_version: {semver or hash — identifies which version of the
        asset is under review}
  published_by: {actor}
  published_at: {ISO 8601}
  participants:
    - {id: participant_1, role: reviewer}
    - {id: participant_2, role: reviewer}
    - {id: participant_3, role: reviewer}
  quorum:
    threshold: majority | supermajority | unanimity
    required_ratio: 0.5 | 0.66 | 1.0           # derived from
        threshold
    abstention_model: neutral                    # default; see
        §Quorum Evaluation
    min_participation_ratio: 0.5                # floor: at least this
        fraction of roster must vote or abstain
  min_duration: {ISO 8601 duration, e.g. P14D}
```

```
  review_closes_at: {ISO 8601}                  # must satisfy:
        review_closes_at >= published_at + min_duration
  notification_channel: {email | github_discussion | slack | in-
        person}
```

Configuration invariant: `review_closes_at >= published_at + min_duration`. If this condition is violated at publication time, the publication is rejected with a `configuration_invalid` error before the review window opens. Equality is permitted — the window may close exactly when `min_duration` elapses.

Phase 2: Comment Collection

Participants submit comments during the review window. The comment set is partitioned at `review_closes_at`:

Gating comment set: comments with `timestamp <= review_closes_at`. These are subject to the `all_gating_comments_dispositioned` convergence check. All must be dispositioned before convergence is possible.

Late comments: comments with `timestamp > review_closes_at`. These are accepted as context for the proposer and future iterations. They do not affect the gating calculation. Exception: if a late comment reveals a material defect, the proposer may invoke `re_open` (see recovery paths) — this is a human decision, not an automatic gate mutation.

```
comment:
  participant: {id}
  timestamp: {ISO 8601}
  content: string
  gating: true | false        # derived: true if timestamp <=
        review_closes_at
  disposition: null | resolved | rejected | acknowledged |
        scope_change
  disposition_rationale: string   # required when disposition is set
```

Disposition rules: - `resolved` — the asset was updated to address the comment - `rejected` — explicitly considered and rejected with rationale - `acknowledged` — noted, addressed in a future iteration (scope management) - `scope_change` — triggers a `spec_modified` event; may expand the asset scope

Asset versioning during review: If comment dispositions result in asset changes, each change is recorded as a new `asset_version`. The rules are:

- **Non-material changes** (clarifications, editorial corrections): preserve vote validity. Votes cast against prior versions remain valid.
- **Material changes** (scope changes, substantive modifications identified by `scope_change` disposition): invalidate all prior votes. The asset is re-published at the new version; votes are reset. The review window resets from the re-publication timestamp with a new `min_duration`.

The proposer classifies whether a change is material. This is an F_H decision — not deterministic. The classification is recorded in the `spec_modified` event.

Phase 3: Voting

Each participant casts one vote per open review cycle. Votes are attached to the `asset_version` at time of casting.

```
vote:
  participant: {id}
  timestamp: {ISO 8601}
  asset_version: {version voted on}
  verdict: approve | reject | abstain
  rationale: string          # required for reject; optional for
        approve and abstain
  conditions: list           # optional — "approve if X is resolved
        before close"
```

Conditional approvals resolve to `approve` if conditions are met at close time, `abstain` otherwise.

Non-response: a rostered participant who neither votes nor abstains before `review_closes_at` is classified as `non_response`. Non-response is not the same as abstention — it is absence, not a deliberate neutral position. Non-response always counts against the participation floor (see §Quorum Evaluation). Whether non-response counts against the approve ratio depends on the abstention model — in the `neutral` model, it does not; in the `counts_against` model, it does.

Phase 4: Quorum Evaluation (deterministic gate)

Five deterministic checks, all required before convergence is attempted:

```
1. min_duration_elapsed:           current_time > published_at +
min_duration
2. review_window_closed:           current_time > review_closes_at
3. participation_threshold_met:    (votes_received + abstain_count) /
roster_size >= min_participation_ratio
4. quorum_reached:                 approve_ratio satisfies threshold
(see formula below)
5. gating_comments_dispositioned:  every comment where gating=true has
non-null disposition
```

Term definitions: - `roster_size`: total number of participants listed in the publication - `votes_received`: participants who cast `approve` or `reject` (excluding abstain and non_response) - `abstain_count`: participants who explicitly cast `abstain` - `non_response_count`: participants who neither voted nor abstained before close - `eligible_votes`: votes_received + abstain_count (participants who responded) - `participation_threshold_met`: eligible_votes / roster_size >= min_participation_ratio

Approve ratio formula (varies by abstention model):

```
abstention_model = neutral (default):
  approve_ratio = approve_votes / (approve_votes + reject_votes)
  # abstentions and non_responses excluded from denominator
```

```
abstention_model = counts_against:
  approve_ratio = approve_votes / roster_size
  # abstentions and non_responses both reduce the ratio
```

Threshold mapping:

```
majority       → approve_ratio > 0.5
supermajority  → approve_ratio >= 0.66
unanimity      → approve_ratio = 1.0
```

Default abstention model: `neutral`. This reflects that abstention is a deliberate "no opinion" — not a vote against. The participation floor (`min_participation_ratio`) prevents the neutral model from being exploited by strategic silence: if too few participants respond, `participation_threshold_met` fails regardless of the approve ratio.

Tie semantics: exactly 0.5 approve_ratio with `majority` threshold → treated as `consensus_failed` with `failure_reason: tie`.

Veto Role

Veto semantics are **deferred to ADR-S-027**. The questions left open by introducing veto in this ADR — whether a veto can be withdrawn, whether it is conditional, whether it must include rationale, whether it applies post-close — expand the semantic surface area beyond the scope of the core CONSENSUS mechanism.

> **Note**: ADR-S-026 was consumed by Named Compositions and Intent Vectors (2026-03-08). The veto extension is ADR-S-027 (pending).

The most common protection cases (architecture review board, lead approver authority) are coverable within this ADR using: - `unanimity` threshold (any reject blocks) - `min_participation_ratio: 1.0` (all roster members must respond) - Named required participants (implementation concern — an implementation may require specific participants to vote, not just any quorum of the roster)

Veto as a named role override of quorum arithmetic is a governance extension, not a prerequisite.

Convergence Outcomes

Positive: `consensus_reached`

```
{
  "event_type": "consensus_reached",
  "timestamp": "{ISO 8601}",
  "project": "{project}",
  "feature": "{REQ-F-*}",
  "edge": "{source}→{target}",
```

```
  "data": {
    "asset_id": "{id}",
    "asset_version": "{version}",
    "quorum_threshold": "majority|supermajority|unanimity",
    "abstention_model": "neutral|counts_against",
    "roster_size": 0,
    "eligible_votes": 0,
    "approve_votes": 0,
    "reject_votes": 0,
    "abstain_votes": 0,
    "non_response_count": 0,
    "approve_ratio": 0.0,
    "participation_ratio": 0.0,
    "gating_comments_total": 0,
    "gating_comments_dispositioned": 0
  }
}
```

Asset proceeds with `stability: ratified` (if paired with RATIFY) or continues to next edge.

Negative: `consensus_failed` (first-class typed outcome)

`consensus_failed` is not "did not converge" — it is a typed terminal state that carries the failure reason and available recovery paths. Each `failure_reason` implies a different recovery.

```
{
  "event_type": "consensus_failed",
  "timestamp": "{ISO 8601}",
  "project": "{project}",
  "feature": "{REQ-F-*}",
  "edge": "{source}→{target}",
  "data": {
    "asset_id": "{id}",
    "asset_version": "{version}",
    "failure_reason": "quorum_not_reached | tie |
        participation_floor_not_met | window_closed_insufficient_votes
        | material_change_reset",
    "roster_size": 0,
    "eligible_votes": 0,
    "approve_votes": 0,
    "reject_votes": 0,
    "abstain_votes": 0,
    "non_response_count": 0,
    "approve_ratio": 0.0,
    "participation_ratio": 0.0,
    "gating_comments_undispositioned": 0,
    "available_paths": ["re_open", "narrow_scope", "abandon"]
  }
}
```

Failure reason guide: - `quorum_not_reached` — participation floor met, but approve ratio below threshold - `tie` — exactly 0.5 with majority threshold - `participation_floor_not_met` — too few participants responded before close - `window_closed_insufficient_votes` — window closed before enough votes were cast (subset of participation failure, surfaced separately for diagnostics) - `material_change_reset` — a material asset change invalidated prior votes; new cycle begins automatically

Three Recovery Paths (F_H decision — proposer selects)

`re_open`: Extend the review window. Notify non-respondents. A new `min_duration` begins from the re-open timestamp. The asset version is unchanged. A new CONSENSUS iteration begins. Appropriate when: insufficient participation, not substantive disagreement.

`narrow_scope`: Fold back to the previous node. Remove contested portions from the asset. Re-publish a reduced proposal at a new `asset_version`. Appropriate when: specific sections are blocked but the majority is agreed. The removed portions become a new candidate asset for a separate CONSENSUS cycle.

`abandon`: Fold back to intent. Close the feature with `convergence_type: consensus_failed`. Appropriate when: fundamental disagreement on the problem, not the solution.

The recovery path is not selected automatically. It is an F_H decision by the proposer. The selection is recorded as a `recovery_path_selected` event. A proposer who selects `re_open` repeatedly without changes to the asset or roster is exhibiting a stuck-delta pattern — homeostasis monitors should surface this after N re-opens.

Event Taxonomy

New event types added to the OL taxonomy:

```
proposal_published        — CONSENSUS opens; includes asset_id,
asset_version, roster, quorum config
comment_received          — per gating comment submitted (gating:
true|false flag)
vote_cast                 — per vote received; includes asset_version
voted on
asset_version_changed     — material or non-material asset change
during open review; includes materiality classification
consensus_reached         — quorum satisfied, participation floor met,
all gating comments dispositioned
consensus_failed          — typed failure with failure_reason and
available_paths
recovery_path_selected    — proposer selects re_open | narrow_scope |
abandon
```

Integration with Existing Model

Evaluator model: unchanged. CONSENSUS is `F_H(roster: N, quorum: rule, participation: floor)`. Three evaluator types remain.

Processing phases: unchanged. CONSENSUS operates at the Conscious phase — persistent ambiguity resolved by multi-stakeholder judgment.

IntentEngine: unchanged. CONSENSUS is one instance of `observer → evaluator → typed_output`. The evaluator is MultiF_H. The output is `consensus_reached` (reflex.log) or `consensus_failed` (escalate).

Profile routing: `standard` and `full` profiles may include CONSENSUS at governance edges. `poc`, `spike`, `hotfix`, `minimal` profiles skip CONSENSUS — they use bare F_H (single human) or no human gate. This is a composition variant, not an exception.

RATIFY relationship: `RATIFY = CONSENSUS + Promote(stability)`. CONSENSUS is the evaluation; RATIFY is CONSENSUS composed with a stability state change. They are separable.

HIGHER_ORDER_FUNCTORS.md: CONSENSUS is defined at §4.4 of the functor library. This ADR is the normative reference for its full operational semantics.

What This Does Not Cover

- **Veto role**: deferred to ADR-S-027 (pending). Coverable in the interim via unanimity threshold or min_participation_ratio: 1.0.
- **Agent-as-participant**: whether an F_P agent can hold a roster position alongside humans. Left explicitly open — requires a separate ADR when the F_P/F_H epoch boundary is formally addressed.
- **Weighted voting**: all participants have equal weight. Weighted quorum is not in scope.
- **Asynchronous comment collection mechanism**: how implementations collect and structure comments is an implementation concern. Implementations must document their collection mechanism.

Consequences

Positive: - Multi-party evaluation is formally modelled with complete semantics. Public review, ADR acceptance, and release gates share the same underlying mechanism. - Consensus failure is a first-class typed outcome. Each `failure_reason` is distinct and maps to a specific recovery path. - Silence is formally distinguished from abstention. The participation floor prevents silent quorum erosion. - The gating comment set is frozen at close time. Late material comments require an explicit human decision to re-open — convergence timing is deterministic. - Asset versioning prevents approval drift across revisions during an open review. - The four-primitive model is preserved. No new evaluator type is introduced.

Negative / trade-offs: - CONSENSUS adds implementation complexity: roster management, comment collection, gating set computation, participation tracking, asset versioning. Implementations must build this or provide a convention. - The `min_duration` lower-bound constraint and the `participation_threshold` introduce two new constraint types not previously in the spec. §5.3 (AI_SDLC_ASSET_GRAPH_MODEL.md) has been updated accordingly. - Material change during an open review resets votes. This is correct governance behaviour, but it adds observable complexity: a single CONSENSUS cycle may contain multiple asset versions in the event log. - `consensus_failed` with `narrow_scope` recovery creates a fold-back path that may produce multiple CONSENSUS cycles for the same intent. This is correct behaviour but adds observable complexity. - The re-open stuck-delta pattern (proposer re-opens repeatedly without change) requires homeostasis monitoring — it is not self-resolving.

References

- AI_SDLC_ASSET_GRAPH_MODEL.md §5.3 — constraint tolerances; lower-bound constraint type (updated)
- specification/core/HIGHER_ORDER_FUNCTORS.md §4.4 — CONSENSUS in the functor library
- specification/core/PROJECTIONS_AND_INVARIANTS.md — profile routing (which profiles include CONSENSUS)
- ADR-S-024 — consensus decision gate (marketplace gating — different concern)
- ADR-S-026 — Named Compositions and Intent Vectors (five-level stack, PLAN/POC/SCHEMA_DISCOVERY/DATA_DISCOVERY, typed gap.intent output)
- ADR-S-027 — veto role extension (pending — deferred from this ADR)
- `.ai-workspace/comments/claude/20260308T090000_STRATEGY_ADR-S-025-CONSENSUS-Functor-Proposal.md` — original proposal
- `.ai-workspace/comments/gemini/20260308T061000_REVIEW_ADR-S-025-CONSENSUS-Functor-Proposal.md` — Gemini review
- `.ai-workspace/comments/codex/20260308T173425_REVIEW_ADR-S-025-CONSENSUS-semantics-and-gaps.md` — Codex review
- `.ai-workspace/comments/claude/20260308T080000_REVIEW_Response-to-Gemini-and-Codex.md` — prior accepted corrections